

Sesiunea 3: Universul Binar și Porțile Logice

Capitolul 1: Sistemul Binar și Hexazecimal - Cum numără mașinile 🖨️	2
1. De ce folosesc mașinile Sistemul Binar? (Baza 2)	3
2. Cum numărăm în Binar? (Valoarea poziției)	3
3. Hexazecimal (Baza 16) - Prietenul Programatorului	4
💡 Analogia QualiAdept: „Tabloul cu 8 întrerupătoare (Byte-ul)”	6
🕵️ Ochiul de Tester: Boundary Value Analysis și „Numerele Magice”	6
🧪 Exercițiu de reflexie	7
Capitolul 2: Reprezentarea Textului - De la ASCII la Unicode și Emoji-uri 📄	8
1. ASCII - Standardul American (Alfabetul pe 7 biți)	8
2. Problema Globalizării și „Turnul Babel”	9
3. Unicode și UTF-8 - Soluția Supremă	9
💡 Analogia QualiAdept: „Meniul Restaurantului Global”	10
🕵️ Ochiul de Tester: Mojibake și Limitele de Caractere	11
Codarea textului este o mină de aur pentru bug-uri și terenul de joacă preferat al unui Software Tester.	11
🧪 Exercițiu de reflexie	12
Capitolul 3: Reprezentarea Imaginilor și Sunetului (Pixeli, Culori, Eșantionare) 🎨🎵	12
1. Reprezentarea Imaginilor - Lumea redusă la puncte (Pixeli)	13
2. Cum dăm culoare unui Pixel? (Modelul RGB)	14
3. Reprezentarea Sunetului - „Felierea” timpului (Eșantionarea)	16
💡 Analogia QualiAdept: „Peisajul prin plasa de țânțari”	17
🕵️ Ochiul de Tester: Artefactele Digitale (Compresia)	17
🧪 Exercițiu de reflexie	18
Capitolul 4: Porțile Logice de Bază (AND, OR, NOT, NAND, NOR, XOR) 🧠⚡	18
1. Ce este un Tabel de Adevăr (Truth Table)?	19
2. Porțile Fundamentale (NOT, AND, OR)	19
3. Porțile Derivate și „Universale” (NAND, NOR)	20

4. Poarta XOR (Exclusive OR) - Decizia Strictă	21
💡 Analogia QualiAdept: „Sistemul de Securitate al Băncii”	23
🕵️ Ochiul de Tester: Tehnica Decision Table Testing (Tabelul de Decizie).....	23
🧪 Exercițiu de reflexie.....	24
Capitolul 5: Sumatoare și Circuite Logice - Cum se adună fizic 1 + 1 🧮	25
1. Cum adunăm în Binar? (Regulile de bază)	25
2. Semi-Sumatorul (The Half-Adder)	25
3. Sumatorul Complet (The Full-Adder) și Efectul de Domino	26
💡 Analogia QualiAdept: „Linia de Asamblare a Găleților”	29
🕵️ Ochiul de Tester: Întârzierea Propagării (Propagation Delay)	29
🧪 Exercițiu de reflexie.....	30
Capitolul 6: Algebra Booleană - Matematica din spatele deciziilor digitale 🧮	30
1. Ce este Algebra Booleană?.....	31
2. Optimizarea și Simplificarea - Cum facem procesoare mai ieftine.....	31
3. Legile lui De Morgan - Magia Inversării.....	33
💡 Analogia QualiAdept: „Regulamentul Clubului”.....	33
🕵️ Ochiul de Tester: Optimizarea Condițiilor de Cod	34
🧪 Exercițiu de reflexie.....	35
Capitolul Bonus: Când matematica binară distruge lumea - Cele mai faimoase bug-uri din istorie 🐛	36
1. Explozia de 500 de milioane de dolari (Racheta Ariane 5 - 1996)	36
2. Sfârșitul universului Pac-Man (Nivelul 256 - 1980)	37
3. Cum a stricat Gangnam Style YouTube-ul (2014)	37
💡 Analogia QualiAdept: „Odometrul mașinii”	38
🕵️ Ochiul de Tester: BVA (Boundary Value Analysis).....	38
🧪 Exercițiu de reflexie: Următorul Sfârșit al Lumii Digitale.....	38

Capitolul 1: Sistemul Binar și Hexazecimal - Cum numără mașinile



🧠 Am lăsat în urmă siliciul și condensatorii.

Știm că la nivel fizic, un computer are doar miliarde de tranzistori care pot fi închisi sau deschiși (Curent / Fără curent).

Dar cum se transformă această rețea imensă de întrerupătoare într-un text, o imagine sau un joc video? Răspunsul stă în cel mai simplu, dar puternic limbaj din univers: Sistemul Binar.

1. De ce folosesc mașinile Sistemul Binar? (Baza 2)

Noi, oamenii, folosim Sistemul Zecimal (Baza 10) cu zece cifre (0-9). De ce? Probabil pentru că am evoluat având 10 degete la mâini.

Un computer, în schimb, are doar „două degete”: tranzistorul PORȚIT (trece curentul) sau OPRIT (nu trece curentul).

- **Binar (Baza 2):** Are doar două „cifre” posibile: **0** și **1**.
- Fiecare 0 sau 1 se numește **Bit** (de la *Binary Digit*). Acesta este atomul informației. Nu există o unitate mai mică de date.
- **Byte-ul (Octetul):** Pentru că un singur bit nu poate exprima mare lucru, computerele le grupează mereu câte 8. Un grup de 8 biți se numește Byte (sau Octet).

2. Cum numărăm în Binar? (Valoarea poziției)

În sistemul nostru cu baza 10, fiecare poziție a unui număr este o putere a lui 10 (Unități, Zeci, Sute, Mii). Exemple: numărul 105 înseamnă 1 Sută + 0 Zeci + 5 Unități.

În sistemul binar (baza 2), pozițiile se dublează de la dreapta la stânga: 1, 2, 4, 8, 16, 32, 64, 128.

- Dacă pe o poziție avem **1**, adunăm acea valoare.
- Dacă avem **0**, o ignorăm.

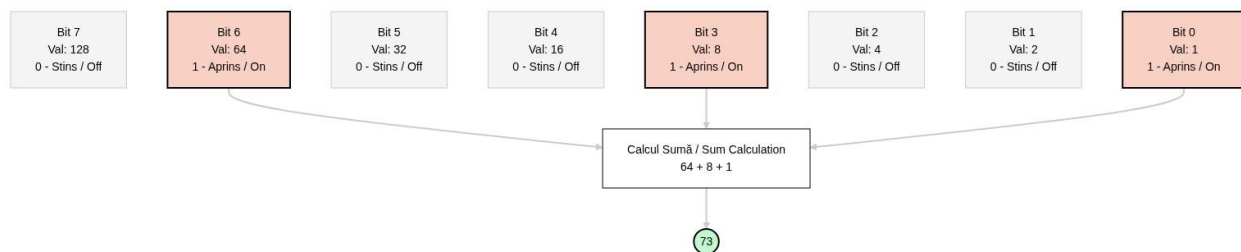


Fig. 1 - Această diagramă ilustrează modul în care numărul binar **01001001** este convertit în numărul zecimal **73** folosind valorile poziționale pe 8 biți (1 octet):

- **Valorile Poziționale:** Fiecare bit dintr-un octet are o valoare zecimală specifică (de la stânga la dreapta: 128, 64, 32, 16, 8, 4, 2, 1).
- **Starea Bitului:** Biții pot fi activați (**1 - Aprins**, evidențiați cu fundal portocaliu) sau dezactivați (**0 - Stins**).
- **Procesul de Calcul:** Se iau în considerare doar valorile poziționale corespunzătoare biților care au valoarea 1. În acest caz, adunăm valorile pentru pozițiile activate: 64 + 8 + 1.
- **Rezultatul Final:** Suma acestor valori oferă echivalentul zecimal, adică **73**.

3. Hexazecimal (Baza 16) - Prietenul Programatorului

Să scrii numere mari în binar este un coșmar pentru ochiul uman. De exemplu, un banal cod de culoare albastru deschis este 01100100 10010101 11101101. Este greu de citit și ușor de greșit.

Pentru a simplifica lucrurile, informaticienii au inventat o „scurtătură”: Sistemul **Hexazecimal**.

- **Ce este?** Este un sistem cu baza 16. Folosește cifrele de la **0 la 9**, iar pentru valorile de la 10 la 15 împrumută primele 6 litere din alfabet: **A, B, C, D, E, F**. (A=10, B=11, C=12, D=13, E=14, F=15)
- **Superputerea Hexazecimalului:** Exact 4 biți de informație pot fi comprimați într-un singur caracter hexazecimal. Un Byte întreg (8 biți) se scrie mereu folosind doar **două** caractere Hex (ex: 11111111 în binar = FF în hexazecimal).

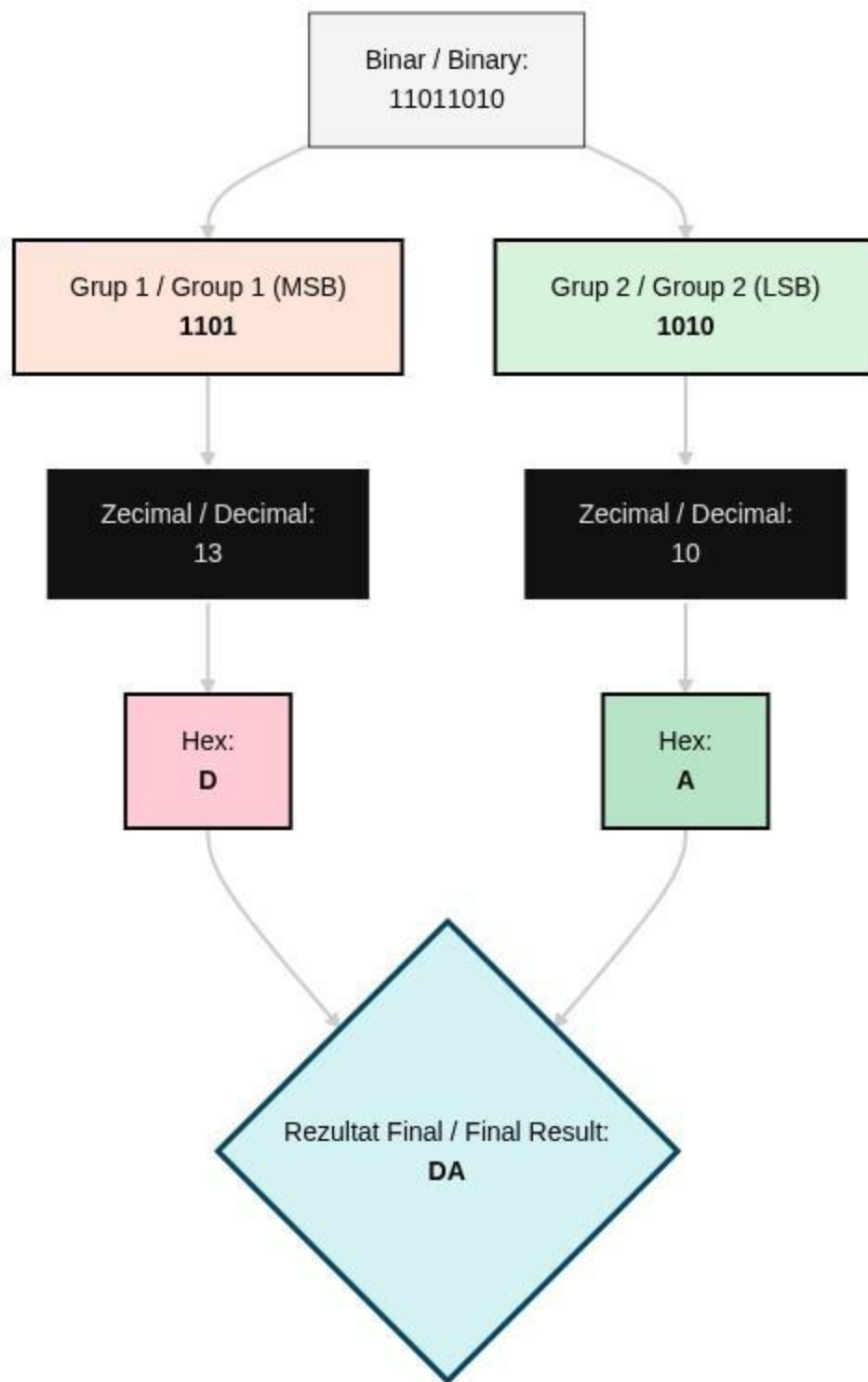


Fig. 2 - Această diagramă ilustrează procesul de comprimare și conversie a unui număr binar de 8 biți (**11011010**) în sistemul hexazecimal (**DA**):

- **Segmentarea (Nibbles):** Numărul binar este împărțit de la dreapta la stânga în grupuri

de câte 4 biți. Astfel, obținem **1101** (Grupul 1) și **1010** (Grupul 2).

- **Conversia Zecimală:** Fiecare grup este convertit individual în valoarea sa zecimală: $8 + 4 + 0 + 1 = 13$ pentru primul grup și $8 + 0 + 2 + 0 = 10$ pentru al doilea.
- **Transpunerea în Hexazecimal:** În sistemul hexazecimal, numerele de la 10 la 15 sunt reprezentate prin litere de la A la F. Prin urmare, zecimalul **13** devine **D**, iar zecimalul **10** devine **A**.
- **Rezultatul Final:** Prin alăturarea celor două caractere obținem codul hexazecimal final: **DA**.

💡 Analogia QualiAdept: „Tabloul cu 8 întrerupătoare (Byte-ul)”

💡 **Imaginează-ți că în fața ta ai un tablou cu 8 întrerupătoare (becuri).** Fiecare bec are un număr scris deasupra lui: 1, 2, 4, 8, 16, 32, 64, 128.

Regula jocului este simplă: poți crea absolut orice număr de la **0 la 255** aprinzând combinația corectă de becuri și adunând numerele lor.

- Vrei să reprezinți numărul **0**? Toate becurile sunt stinse (00000000).
- Vrei numărul **3**? Aprinzi becul „2” și becul „1” (00000011).
- Vrei numărul maxim, **255**? Aprinzi toate becurile (11111111). Aceasta este limita absolută a unui singur Byte!

🕵️ Ochiul de Tester: Boundary Value Analysis și „Numerele Magice”

📄 Pentru un Software Tester, înțelegerea limitelor binare este esențială. Bug-urile se ascund adesea la granițele memoriei.

- **„Numerele Magice” din IT:** Ca tester, vei vedea des anumite numere care cauzează erori în aplicații: **255** (limita unui byte), **65.535** (limita a doi bytes), **2.147.483.647** (limita a patru bytes / 32-bit integer).

- **Aplicație practică (BVA):** Dacă testezi un câmp dintr-un formular unde utilizatorul trebuie să introducă vârsta, un tester educat nu va testa doar valorile 18 și 99. Va testa neapărat **255** și **256**. De ce? Pentru că dacă programatorul a alocat un singur Byte în baza de date pentru vârstă, introducerea numărului 256 va provoca o eroare de tip *Integer Overflow* (aplicația se va reseta la 0, făcând utilizatorul un „nou-născut”).

Exercițiu de reflexie

Uită-te la routerul tău de internet sau la setările telefonului tău. Vei vedea ceva numit **Adresa MAC** (ex: 00:1A:2B:3C:4D:5E).

De asemenea, dacă folosești un program de grafică (Paint, Photoshop), vei vedea că albul perfect are codul de culoare #FFFFFF.

- *Ce sistem de numerație folosesc ambele exemple de mai sus? De ce crezi că industria a ales acest format în locul zecimalului obișnuit sau al binarului uriaș?*

Capitolul 2: Reprezentarea Textului -

De la ASCII la Unicode și Emoji-uri 📄

🧐 Știm că la nivel fundamental, un procesor înțelege doar zerouri și de unari. El nu știe ce este o literă, un cuvânt sau o propoziție. Prin urmare, pentru a procesa text, primii informaticieni au trebuit să inventeze un sistem universal de traducere: un dicționar uriaș în care fiecare literă din alfabet este asociată cu un număr unic.

1. ASCII - Standardul American (Alfabetul pe 7 biți)

În anii '60, computerele aveau nevoie de un limbaj comun pentru a face schimb de texte. Așa a apărut **ASCII** (American Standard Code for Information Interchange).

- **Cum funcționează:** Folosește exact **7 biți** pentru fiecare caracter. Deoarece $2^7 = 128$, tabelul ASCII conține exact 128 de poziții unice (de la 0 la 127).
- **Ce conține?** Alfabetul englezesc (litere mari și mici), cifrele de la 0 la 9, semnele de punctuație de bază (virgulă, punct, spațiu) și câteva comenzi invizibile de control (cum ar fi „Enter” / New Line).
- **Exemplu:** Când apeși litera „A” pe tastatură, computerul nu vede o literă. El caută în tabelul ASCII, vede că litera „A” corespunde numărului **65**, și o salvează în memorie sub forma octetului binar 01000001.



Fig. 3 - Această diagramă explică modul în care calculatorul traduce o acțiune fizică (apăsarea unei taste) în date codificate pe care procesorul și memoria RAM le pot înțelege, folosind standardul **ASCII**:

- **Evenimentul de Intrare:** Utilizatorul apasă tasta **A** (majusculă) pe tastatură. Controlerul tastaturii trimite un semnal către sistem.

- **Căutarea în Tabelul ASCII:** Sistemul de operare traduce acest semnal folosind tabelul standard ASCII, unde fiecărui caracter îi corespunde un număr unic. Pentru caracterul **A**, valoarea zecimală asociată este **65**.
- **Stocarea în Memorie:** Deoarece circuitele electronice funcționează doar cu stări de tip „pornit/oprit”, numărul zecimal **65** este convertit instantaneu în cod binar pe 8 biți (1 octet): **01000001**, formă sub care este salvat în memoria RAM.

2. Problema Globalizării și „Turnul Babel”

ASCII a fost perfect pentru limba engleză. Dar ce facem cu francezii care aveau nevoie de é, cu românii care aveau nevoie de ș și ț, sau cu rușii care foloseau alfabetul chirilic? Aceștia nu încăpeau în cele 128 de spații ASCII.

- **Code Pages (Paginile de Cod):** Pentru a rezolva problema, s-a folosit și al 8-lea bit din Byte (crescând spațiile la 256). Primele 128 au rămas ASCII standard, dar următoarele 128 au fost personalizate în funcție de țară.
- **Haosul:** Un document rusesc deschis pe un computer francez arăta ca niște hieroglife indescifrabile, pentru că poziția 150 însemna o literă chirilică în Rusia, dar o literă cu accent în Franța. Internetul era plin de texte stricate.

3. Unicode și UTF-8 - Soluția Supremă

Pentru a repara acest haos global, giganții tehnologici s-au unit și au creat **Unicode**.

- **Unicode:** Este un dicționar universal masiv. Scopul lui este să asigneze un identificator numeric unic (numit *Code Point*) pentru absolut fiecare caracter din absolut orice limbă umană (trecută și prezentă), plus simboluri matematice și **Emoji-uri** (🚀, 😊).

Dar cum salvăm aceste numere uriașe în memorie fără a irosi spațiu? Aici intervine **UTF-8**.

- **UTF-8 (Unicode Transformation Format - 8-bit):** Este un sistem de codare inteligent, de *lungime variabilă*. Este standardul absolut pe internetul de astăzi.
 - Dacă un caracter este o literă englezească standard (cum ar fi 'A'), UTF-8 folosește doar **1 Byte** (rămâne perfect compatibil retroactiv cu vechiul ASCII).

- Dacă este un caracter special (precum 'ș', '€' sau o literă arabă), folosește dinamic **2 sau 3 Bytes**.
- Dacă este un Emoji complex (precum '🚀'), folosește **4 Bytes**.

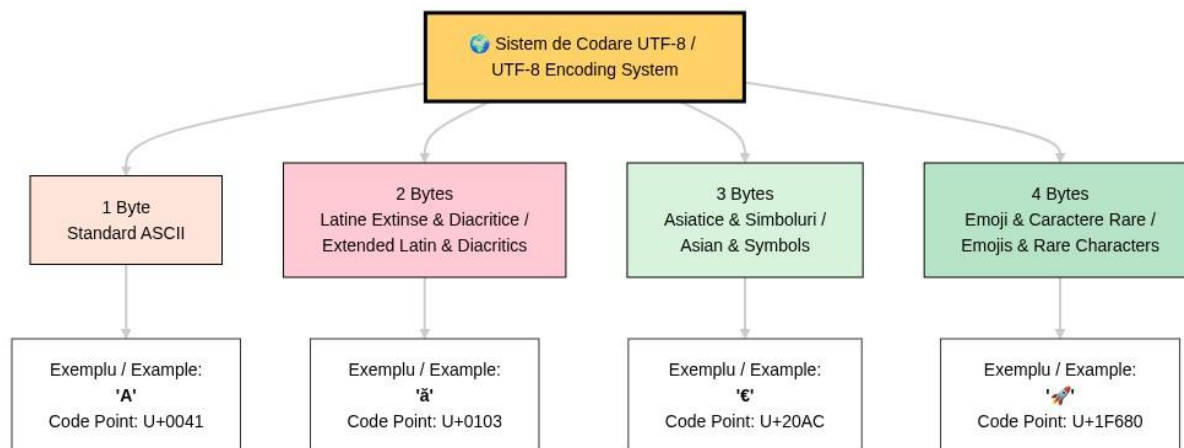


Fig. 4 - Această diagramă ilustrează caracteristica principală a standardului **UTF-8**, și anume **codarea cu lungime variabilă** (folosind între 1 și 4 octeți pentru fiecare caracter):

- **1 Byte (ASCII):** Caracterele din alfabetul englez de bază, cifrele și simbolurile standard ocupă doar 8 biți (1 octet). Sunt identice cu vechiul standard ASCII, asigurând compatibilitate retroactivă completă.
- **2 Bytes:** Folosiți pentru caractere din alfabetul european extins, inclusiv diacriticele limbii române (cum este litera *ă*).
- **3 Bytes:** Alocați pentru majoritatea limbilor asiatice (chineză, japoneză, coreeană) și pentru simboluri matematice sau monetare complexe (precum simbolul Euro €).
- **4 Bytes:** Rezervați pentru caractere foarte rare, scrieri istorice și colecția modernă de simboluri grafice folosite zilnic (cum este emoji-ul rachetă 🚀).

Notă: Spre deosebire de diagramele anterioare, valorile numerice din paranteze au fost înlocuite cu **Code Points** (ex: **U+0103**), reprezentarea hexazecimală oficială din standardul Unicode pentru fiecare caracter.

💡 Analogia QualiAdept: „Meniul Restaurantului Global”

💡 **Imaginează-ți un meniu uriaș într-un restaurant global (Unicode).** Fiecare preparat are un număr unic. Numărul 65 este mereu un Burger ('A'). Numărul 128640 este mereu un Platou Sushi ('🍣'). **UTF-8 este carnețelul ospătarului.** Dacă comanzi un Burger (număr mic), scrie pe o singură foaie de hârtie (1 Byte). Dacă comanzi Platoul Sushi (număr uriaș), prinde cu capsatorul 4 foile împreună pentru a avea loc să scrie tot codul (4 Bytes). Astfel, nu irosește niciodată hârtie degeaba pentru comenzile simple!

🕵️ Ochiul de Tester: Mojibake și Limitele de Caractere

Codarea textului este o mină de aur pentru bug-uri și terenul de joacă preferat al unui Software Tester.

- **Mojibake (Textul Stricat):** Dacă vezi vreodată un site unde textul arată gen **SalutĂlume** în loc de **Salută lume**, este un bug de codare. Baza de date a salvat textul în UTF-8 (folosind 2 bytes pentru 'ă'), dar browserul web a încercat să îl citească folosind un format vechi (care citește byte cu byte). Este datoria testerului să verifice că encodarea este setată uniform pe UTF-8 pe tot stack-ul (Bază de date -> Backend -> Frontend).
- **Bug-ul de Twitter (Lungimea Emoji-urilor):** Dacă un câmp din baza de date are o limită de „10 caractere”, ce se întâmplă dacă utilizatorul introduce 10 Emoji-uri? Deoarece un Emoji ocupă 4 Bytes, 10 Emoji-uri înseamnă 40 Bytes de date. Dacă programatorul nu a anticipat asta și a alocat doar 10 Bytes, baza de date va da eroare sau va trunchia (tăia) datele. Un tester cu experiență inserează mereu emoji-uri în câmpurile de text pentru a vedea cum gestionează sistemul datele cu lungime variabilă!

Exercițiu de reflexie

Știai că mesajele SMS standard au o limită strictă de 160 de caractere? Totuși, dacă scrii un SMS lung și introduci o singură literă cu diacritice (precum ș sau ț), telefonul te va avertiza brusc că limita a scăzut la 70 de caractere.

- *Pe baza a ceea ce am învățat despre ASCII vs. Unicode (care folosește mai mulți biți per caracter), de ce crezi că rețeaua de telecomunicații îți scade drastic limita de caractere în secunda în care folosești o diacritică?*

Capitolul 3: Reprezentarea Imaginilor și Sunetului (Pixeli, Culori,

Eșantionare) 🎧 🎵

🎧 Știm cum să salvăm text în binar. Dar ce facem cu lumea fizică din jurul nostru? Lumea reală (o pictură, vocea ta) este continuă, infinit de complexă. Computerele, în schimb, sunt mașinării finite, discrete (folosesc doar 0 și 1). Pentru a aduce lumea reală în computer, trebuie să o „tăiem” în bucățele foarte mici. Acesta este procesul de digitalizare.

1. Reprezentarea Imaginilor - Lumea redusă la puncte (Pixeli)

O fotografie tipărită pare perfect netedă. Dar dacă o pui sub un microscop, vei vedea o tranziție continuă de culori (informație *analogică*). Pentru a o pune pe ecran, trebuie să o transformăm în informație *digitală*.

- **Pixelul (Picture Element):** Soluția este să suprapunem o grilă (un tabel) invizibilă peste imagine. Fiecare pătrățel minuscul rezultat se numește **Pixel**. Aceasta este cea mai mică unitate a unei imagini digitale.
- **Regula de Aur:** Spre deosebire de un tablou real, un pixel nu poate avea două culori sau nuanțe diferite în interiorul său. **Un pixel poate avea o singură culoare perfect uniformă.**

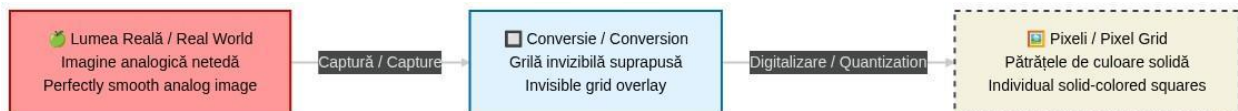


Fig. 5 - Această diagramă ilustrează conceptul de **rezoluție** și modul în care o imagine din lumea reală este transformată în date digitale prin procesul de eșantionare spațială (pixelare):

- **Lumea Reală (Imagine Analogică):** În natură, formele și culorile sunt continue și perfect netede (analogice). Un măr sau un peisaj nu are „marginii zimțate” sau puncte vizibile, oricât de aproape am privi.
- **Conversia Digitală:** În momentul în care facem o fotografie sau scanăm un obiect, senzorul aparatului împarte acea scenă continuă prin suprapunerea unei grile imaginare

de linii verticale și orizontale.

- **Imaginea Digitală (Pixeli):** Rezultatul final este o matrice de **pixeli** (picture elements). Fiecare pătrățel din grilă poate stoca doar o singură culoare solidă, calculată ca o medie a zonei respective. Rezoluția imaginii este determinată de densitatea acestei grile: cu cât grila are mai multe pătrățele (pixeli), cu atât imaginea digitală va fi mai fidelă și mai apropiată de realitate.

2. Cum dăm culoare unui Pixel? (Modelul RGB)

Am obținut o grilă de pixeli, dar cum salvăm culoarea fiecărui pătrățel folosind doar 0 și 1? Inginerii au inventat o metodă bazată pe modul în care ochiul uman percepe lumina.

- **RGB (Red, Green, Blue - Roșu, Verde, Albastru):** Acestea sunt culorile primare în spectrul luminos. Aproape orice culoare pe care o vezi pe un ecran este formată dintr-un amestec al acestor trei lumini.
- **Cât spațiu ocupă o culoare?** Standardul industriei este de a folosi **1 Byte (8 biți)** pentru fiecare culoare primară, dintr-un singur pixel.
 - Deoarece un Byte are 256 de valori posibile (de la 0 la 255), înseamnă că putem avea 256 de intensități de Roșu, 256 de Verde și 256 de Albastru.
- **Culoarea finală a unui pixel ocupă 3 Bytes (24 de biți).**
 - Exemplu: (Roșu: 255, Verde: 0, Albastru: 0) înseamnă Roșu Pur.
 - Exemplu: (Roșu: 0, Verde: 0, Albastru: 0) înseamnă Negru Absolut.
 - Exemplu: (Roșu: 255, Verde: 255, Albastru: 255) înseamnă Alb Pur.
- **Matematica Culorilor:** Dacă înmulțim intensitățile ($256 \times 256 \times 256$), descoperim că modelul standard RGB poate genera **peste 16,7 milioane de culori diferite**, toate salvate ca simple numere binare!

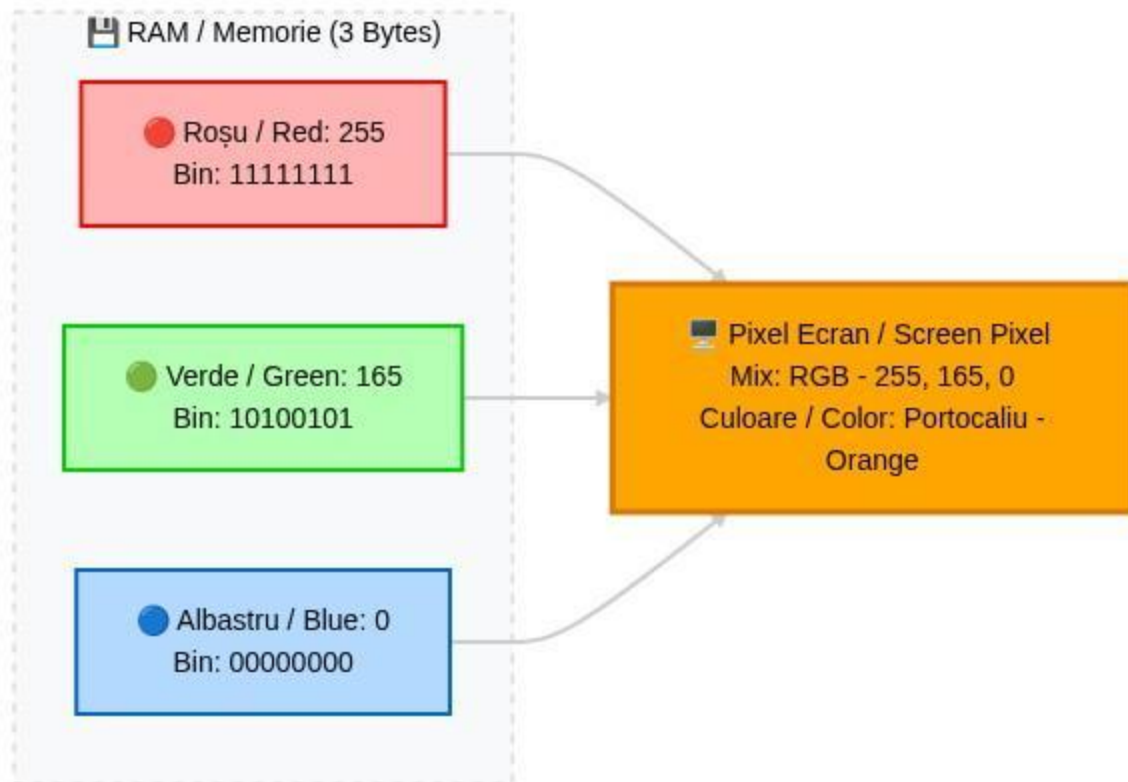


Fig. 6 - Această diagramă ilustrează modul în care ecranele digitale folosesc modelul **RGB (Red, Green, Blue)** pentru a genera o culoare specifică pentru fiecare pixel, utilizând **3 octeți (24 de biți)** de date:

- **Stocarea în RAM (Canalele de Culoare):** Fiecare pixel de pe ecran este controlat de 3 sub-pixeli (fizici sau logici). Memoria alocă exact 1 octet (8 biți) pentru fiecare culoare primară, oferind o plajă de valori de la 0 (stins complet) la 255 (intensitate maximă). În exemplul nostru:
 - **Roșu:** Activat la maximum (255 sau binar 11111111).
 - **Verde:** Activat la o intensitate medie-înaltă (165 sau binar 10100101).
 - **Albastru:** Oprit complet (0 sau binar 00000000).
- **Mixul Fizic:** Lumina emisă de cele trei canale se combină direct pe ecran.
- **Rezultatul Vizual:** Combinația hardware a acestor intensități specifice pixelului generează pentru ochiul uman culoarea **Portocaliu**.

3. Reprezentarea Sunetului - „Felierea” timpului (Eșantionarea)

Lumina poate fi împărțită într-o grilă statică (pixeli). Dar sunetul este o undă fizică (presiunea aerului) care vibrează continuu în timp. Cum o „băgăm” într-un fișier mp3?

- **ADC (Analog-to-Digital Converter):** Când vorbești la microfon, unda sonoră generează un voltaj variabil, continuu. Computerul trebuie să „înghețe” acel voltaj și să-l transforme într-un număr.
- **Eșantionarea (Sampling Rate):** Deoarece computerul nu poate salva o undă continuă, el o „pozează” (ia o mostră sau un *eșantion*) de mii de ori pe secundă.
 - Gândește-te la cum este făcut un film dintr-o rolă cu mii de poze statice rulate rapid. Același lucru se întâmplă cu sunetul.
- **Standardul CD-ului:** Pentru ca sunetul digital să pară perfect continuu urechii umane, industria (pe CD-urile audio obișnuite) a decis să ia **44.100 de eșantioane în fiecare secundă (44.1 kHz)**.

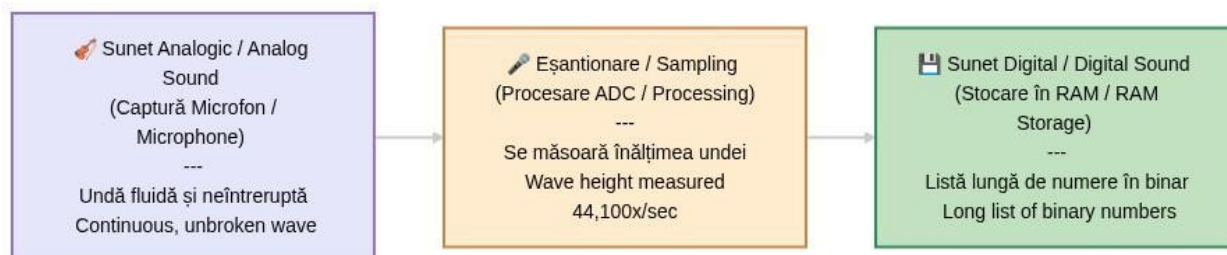


Fig. 7 - Această diagramă ilustrează conceptul de **eșantionare (sampling)**, procesul prin care o undă acustică din lumea fizică este transformată în date audio digitale (cum ar fi un fișier MP3 sau WAV):

- **Sunetul Analogic:** În natură, sunetul (cum este cel produs de o vioară) circulă sub formă de unde de presiune continue și fluide. Nu există pauze sau trepte abrupte în aerul care vibrează.
- **Procesul de Eșantionare:** Deoarece calculatorul nu poate stoca o infinitate de stări fluide, procesul de digitalizare folosește un microfon și un convertor analog-digital (ADC). Acesta „fotografiază” sau măsoară înălțimea undei de mii de ori pe secundă. Standardul pentru calitate CD este de **44.100 de ori pe secundă (44.1 kHz)**.

- **Sunetul Digital:** Rezultatul final salvat în memorie nu mai este o undă, ci o simplă listă de numere binare (0 și 1). La redare, placa de sunet va uni aceste puncte numerice pentru a reconstitui unda originală și a o trimite către difuzoare.

💡 Analogia QualiAdept: „Peisajul prin plasa de țânțari”

💡 **Imaginile (Pixeli):** Imaginează-ți că te uiți la un munte perfect prin gaura cheii (Lumea analogică). Acum, pune o plasă de țânțari în fața aceluia peisaj. Pentru a-l redesena pe caiet, trebuie să umpli fiecare pătrățel al plasei cu o singură culoare dominantă. Cu cât plasa este mai deasă (Rezoluție mai mare), cu atât desenul din caiet va semăna mai mult cu muntele real.

Sunetul (Eșantionarea): Imaginează-ți un dirijor care mișcă bagheta continuu și fluid sus-jos. Pentru a te învăța mișcarea prin mesaje text, el nu-ți poate trimite un videoclip. Așa că el îți trimite un SMS în fiecare milisecundă: *"Acum mâna e la 10cm! Acum e la 12cm! Acum la 14cm!"*. Dacă citești acele numere (eșantioane) foarte repede, mintea ta va reconstitui mișcarea fluidă a baghetei. Acesta este un fișier audio!

🕵️ Ochiul de Tester: Artefactele Digitale (Compresia)

💡 🎯 💰 📊 📄 Un tester cu ochi aprofundat știe că transformarea din analog în digital ocupă un spațiu enorm.

- **Problema Fișierelor RAW:** O poză 4K are peste 8 milioane de pixeli. Dacă fiecare are nevoie de 3 Bytes, o singură poză nefăcută (Raw) ocupă 24 de Megabytes! O melodie necompresată ocupă 10 Megabytes pentru fiecare minut.
- **Compresia (JPEG, MP3):** Pentru ca internetul să funcționeze, algoritmi intervin și „aruncă” (șterg) inteligent din pixeli sau din nuanțele sunetului (compresie *Lossy*), ghicind că ochiul/urechea umană nu va observa.

- **Atenția la Detalii:** Un QA de frontend va depista imediat „artefactele de compresie” (pixeli pătrățoși și urâți în jurul textului pe un banner web), care apar atunci când poza a fost micșorată prea agresiv de programatori pentru a face site-ul să se încarce mai repede.

Exercițiu de reflexie

Imaginați-vă că sunteți echipa de dezvoltare pentru vechiul joc „Super Mario Bros” (lansat în anii '80 pe sistemul Nintendo NES, pe 8 biți).

Consolele vechi nu aveau memorie RAM (aveau doar câțiva Kilobytes).

- *Dacă știm că în RGB o singură culoare complexă ocupă aproape 3 Bytes, de ce credeți că fața lui Mario, cerul și norii din acel joc foloseau doar aproximativ 4 până la 16 culori foarte simple în total, refolosindu-le constant? (Ce limitare fizică dictează nivelul graficii?)*

Capitolul 4: Porțile Logice de Bază (AND, OR, NOT, NAND, NOR, XOR)



🧠 Până acum am transformat totul în numere (biți de 0 și 1). Dar ce face procesorul cu ele? Cunoaștem din Sesiunea 1 că tranzistorii sunt ca niște întrerupătoare. Dacă legăm aceste întrerupătoare între ele în moduri specifice, creăm „Porți Logice” (Logic Gates). Ele sunt elementele de bază ale „gândirii” mașinii. Orice program complex, de la Windows la cel mai avansat joc 3D, este în esență doar o cascadă masivă a acestor decizii simple.

1. Ce este un Tabel de Adevăr (Truth Table)?

Pentru a înțelege cum gândește o poartă logică, informaticienii folosesc un **Tabel de Adevăr**. Este pur și simplu un tabel care arată toate combinațiile posibile de semnale care pot intra în poartă (Input) și ce rezultat va ieși (Output).

- 0 = Fals, Oprit, Fără curent.
- 1 = Adevărat, Pornit, Curent prezent.

2. Porțile Fundamentale (NOT, AND, OR)

Acestea sunt cele trei piese de bază ale universului digital.

- **Poarta NOT (Inversorul):** Este cel mai simplu circuit. Are un singur Input. Orice intră în el, este răsturnat. Dacă intră 1, iese 0. Dacă intră 0, iese 1.
- **Poarta AND (ȘI):** Are două (sau mai multe) Input-uri. Regula este strictă: Output-ul este 1 **DOAR DACĂ absolut toate** Input-urile sunt 1. Dacă măcar un Input este 0, rezultatul va fi 0.
- **Poarta OR (SAU):** Are două (sau mai multe) Input-uri. Regula este relaxată: Output-ul este 1 dacă **CEL PUȚIN UN** Input este 1. Rezultatul este 0 doar dacă toate intrările sunt 0.

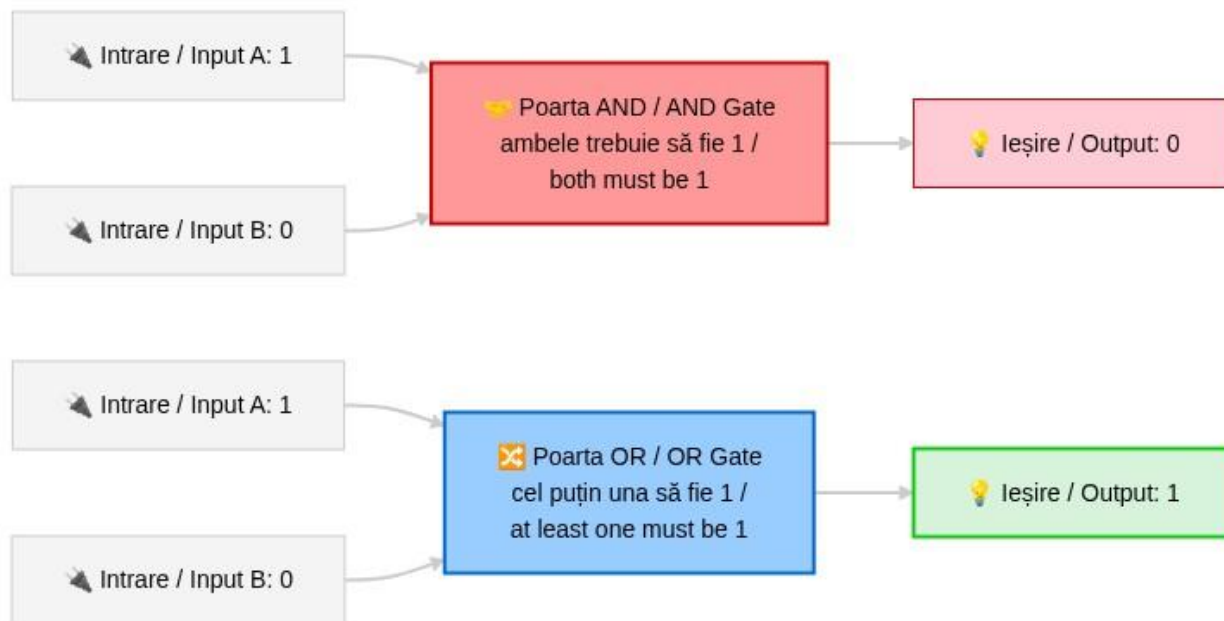


Fig. 8 - Această diagramă compară comportamentul fundamental al celor două porți logice de bază utilizate în circuitele digitale și în programare (**AND** și **OR**), folosind aceleași valori de intrare (1 și 0):

- **Poarta AND (ȘI):** Este o poartă restrictivă. Funcționează după regula: *ieșirea este 1 doar dacă toate intrările sunt 1*. În exemplul nostru, deoarece Intrarea B este 0 (dezactivată), poarta blochează semnalul, iar ieșirea finală devine 0.
- **Poarta OR (SAU):** Este o poartă permisivă. Funcționează după regula: *ieșirea este 1 dacă cel puțin o singură intrare este 1*. În acest caz, deoarece avem valoarea 1 pe Intrarea A, condiția este îndeplinită cu succes, iar ieșirea devine 1 (activată, evidențiată cu verde).

3. Porțile Derivate și „Universale” (NAND, NOR)

Dacă adăugăm un inversor (NOT) la ieșirea porților de bază, obținem rezultatele opuse.

- **NAND (NOT-AND):** Este opusul lui AND. Iese 0 **doar dacă** ambele intrări sunt 1. În rest, iese mereu 1.
 - *Secretul Industriei:* Poarta NAND este numită **poartă universală**. Din ea poți construi orice altă poartă (AND, OR, NOT). De aceea memoriile SSD se numesc „NAND Flash”

– sunt construite fizic dintr-o mare imensă de astfel de porți!

- **NOR (NOT-OR):** Este opusul lui OR. Iese 1 **doar dacă** ambele intrări sunt 0.

4. Poarta XOR (Exclusive OR) - Decizia Strictă

Aceasta este poate cea mai importantă poartă pentru a înțelege cum face un computer matematică (cum adună).

- **Regula XOR (SAU Exclusiv):** Output-ul este 1 **DACĂ ȘI NUMAI DACĂ** intrările sunt diferite (un 1 și un 0). Dacă intrările sunt la fel (ambele 1 sau ambele 0), Output-ul este 0. Este alegerea clasică „ori una, ori alta, dar nu ambele”.

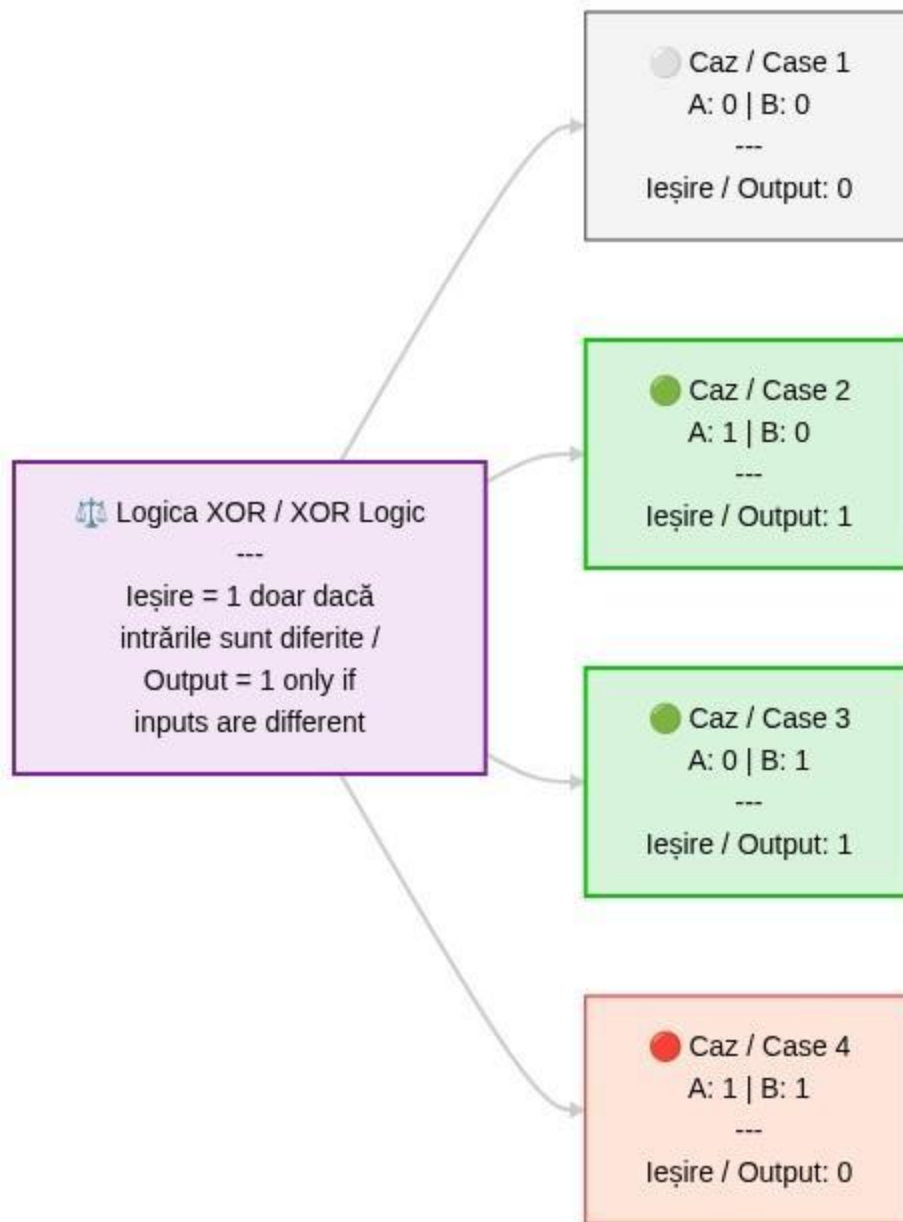


Fig. 9 - Această diagramă reprezintă comportamentul porții logice **XOR (eXclusive OR / SAU Exclusiv)**, esențială în circuitele de calcul (cum ar fi adunatoarele binare) și în criptografie:

- **Regula de Aur:** Poarta XOR verifică exclusivitatea. Ea dă la ieșire valoarea **1** (Adevărat) **doar dacă intrările sale sunt diferite între ele** (una este activată și cealaltă dezactivată). Dacă intrările sunt identice, ieșirea va fi întotdeauna **0**.
- **Analiza Scenariilor:**

- **Cazurile 1 și 4 (leșire 0):** Intrările sunt identice (0 și 0 sau 1 și 1). Deoarece nu există o diferență între ele, poarta se dezactivează.
- **Cazurile 2 și 3 (leșire 1):** Intrările sunt opuse (1 și 0 sau 0 și 1). Condiția de exclusivitate este îndeplinită cu succes, iar rezultatul este activat (evidențiat cu verde).

Analogia QualiAdept: „Sistemul de Securitate al Băncii”

Să transpunem logica în viața reală:

- **Poarta AND (Seiful):** Pentru a deschide ușa seifului (Output 1), managerul trebuie să întoarcă cheia lui (Input A = 1) **ȘI** gardianul trebuie să întoarcă cheia lui (Input B = 1). Dacă unul lipsește, seiful rămâne închis (0).
- **Poarta OR (Alarma de incendiu):** Pentru ca alarma să sune (Output 1), este suficient ca senzorul din hol să detecteze fum (Input A = 1) **SAU** senzorul din birou să detecteze temperatură mare (Input B = 1). Nu trebuie să ia foc ambele camere simultan!
- **Poarta XOR (Lumina de pe hol):** Ai un întrerupător la capătul holului și unul la celălalt capăt. Dacă apeși doar pe unul (sunt diferite), lumina se aprinde. Dacă ambele sunt în aceeași poziție (ambele sus sau ambele jos), lumina este stinsă.

Ochiul de Tester: Tehnica Decision Table Testing (Tabelul de Decizie)

 Pentru un inginer QA, acest capitol nu este doar teorie hardware, ci însăși baza testării software avansate!

- **De la Hardware la QA:** Când testezi o aplicație care are reguli de business complexe, cum te asiguri că nu ai ratat niciun scenariu? Folosești tehnica **Decision Table Testing**,

care este o copie directă a Tabelelor de Adevăr din logica digitală!

- **Exemplu:** Să presupunem că un site web are regula: *"Oferim transport gratuit (Output) doar clienților VIP (Input A) care comandă peste 500 RON (Input B)."*
- Ochiul de Tester știe că aceasta este o **Poartă AND**. Prin urmare, el va crea în Jira un tabel cu fix 4 teste (combinațiile posibile):
 - Nu e VIP (0) + Comandă < 500 (0) -> Fără transport gratuit (0)
 - Este VIP (1) + Comandă < 500 (0) -> Fără transport gratuit (0)
 - Nu e VIP (0) + Comandă > 500 (1) -> Fără transport gratuit (0)
 - Este VIP (1) + Comandă > 500 (1) -> Transport Gratuit (1) (*Singurul test care declanșează regula!*).

Exercițiu de reflexie

Imaginează-ți sistemul de airbag al unei mașini.

Ai doi senzori:

- Senzorul de impact din bara față.
- Senzorul de greutate din scaunul pasagerului (care îți spune dacă stă cineva acolo).

Dacă sistemul trebuie să decidă dacă să declanșeze (Output 1) sau nu airbag-ul pasagerului în caz de accident, ce fel de poartă logică ar trebui să folosească inginerii între cei doi senzori? AND, OR sau XOR? Argumentați alegerea.

Capitolul 5: Sumatoare și Circuite Logice - Cum se adună fizic $1 + 1$

🧠 Acum știm ce sunt porțile logice (AND, OR, XOR). Ele iau decizii simple pe baza curentului electric. Dar cum ajungem de la o simplă poartă la un calculator care poate rula simulări fizice complexe? Secretul stă în combinarea lor pentru a crea „Circuite Logice”. Cel mai important și faimos circuit este Sumatorul (Adder) – mașină fizică ce știe să adune două numere.

1. Cum adunăm în Binar? (Regulile de bază)

Înainte să construim circuitul, trebuie să ne amintim cum adunăm numere pe hârtie, dar folosind doar 0 și 1:

- $0 + 0 = 0$
- $1 + 0 = 1$
- $0 + 1 = 1$
- $1 + 1 = 2$? Nu! În binar nu există cifra 2. Rezultatul este 0, și „ținem 1 în minte” (îl dăm mai departe pe coloana următoare). Acel 1 ținut în minte se numește **Carry** (Transport).

2. Semi-Sumatorul (The Half-Adder)

Vrem un circuit care să adune doi biți (Intrarea A și Intrarea B).

Avem nevoie de două ieșiri: **Suma (S)** și ce ținem în minte, **Carry (C)**.

Dacă ne uităm atent la regulile de mai sus cu ochi de informatician, descoperim ceva magic:

- **Suma** se comportă exact ca o **Poartă XOR** (iese 1 doar dacă intrările sunt diferite).
- **Carry-ul** se comportă exact ca o **Poartă AND** (avem 1 în minte DOAR DACĂ adunăm 1 cu 1).

Așadar, dacă legăm firele intrărilor A și B simultan la o poartă XOR și la o poartă AND, am creat

un circuit care știe matematică: **Semi-Sumatorul**.

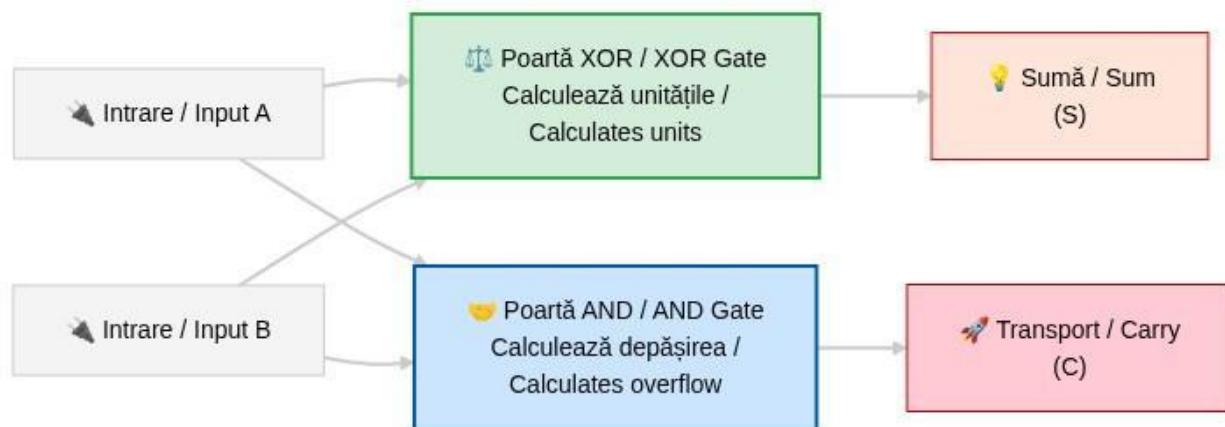


Fig. 10 - Această diagramă ilustrează arhitectura unui **Circuit Semi-Sumator (Half-Adder)**, blocul logic elementar din interiorul procesorului (ALU) responsabil pentru adunarea a doi biți individuali (A și B):

- **Intrările (Inputs):** Circuitul primește două valori binare (0 sau 1) pe care dorește să le adune electronic.
- **Calculul Sumorării (Poarta XOR):** Poarta XOR se ocupă de generarea bitului de **Sumă (S)**. Urmând tabela sa de adevăr, dacă adunăm $1 + 1$, rezultatul pe acest canal este 0 (deoarece în binar $1 + 1 = 10$). Dacă adunăm $1 + 0$, suma va fi 1.
- **Calculul Transportului (Poarta AND):** Poarta AND se ocupă de generarea bitului de **Transport (C - Carry)**. Acesta reprezintă valoarea care „se tunde” către ordinul următor atunci când ambele intrări sunt 1 ($1 + 1$ generează un transport de 1).
- **Limitare:** Se numește „semi-sumator” deoarece poate aduna doar două intrări și nu are capacitatea de a primi un bit de transport generat de un circuit anterior (spre deosebire de un *Full-Adder*).

3. Sumatorul Complet (The Full-Adder) și Efectul

de Domino

Semi-Sumatorul este grozav, dar are o problemă: nu știe să primească un număr „din minte” de la o operație anterioară. Poate aduna doar prima coloană de numere.

Pentru a aduna numere mari (ex: pe 8 biți), avem nevoie de un **Sumator Complet (Full-Adder)**. Acesta primește 3 intrări: Bitul A, Bitul B și Carry-ul de la coloana anterioară (Carry-In).

Se construiește simplu: legând două Semi-Sumatoare împreună cu o poartă OR.

- **Ripple Carry Adder (Sumator în cascadă):** Dacă vrem să adunăm două numere uriașe de 8 biți (1 Byte), punem 8 Sumatoare Complete în linie. Fiecare își face calculul și dă „Carry-ul” (ce ține în minte) mai departe vecinului său din stânga.

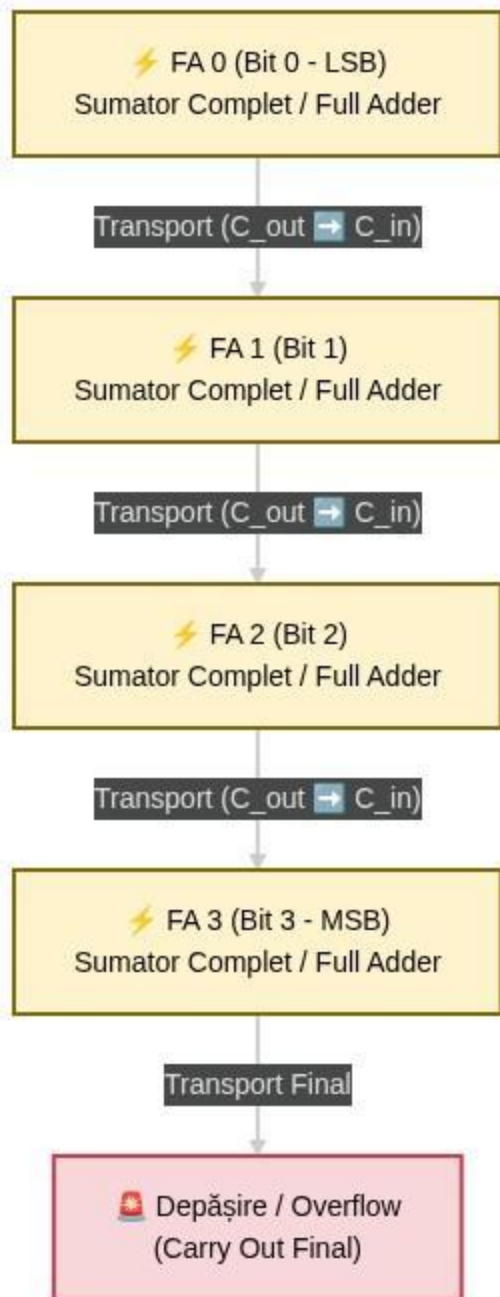


Fig. 11 - Această diagramă ilustrează funcționarea unui **Sumator în cascadă pe 4 Biți (Ripple Carry Adder)**, un circuit digital care permite adunarea a două numere binare de câte 4 biți fiecare:

- **Suma pe biți (Full Adders):** Circuitul este compus din 4 blocuri de tip *Full Adder* (Sumator Complet), numerotate de la FA 0 la FA 3. Fiecare bloc adună o singură coloană de biți (de exemplu, bitul 0 de la primul număr cu bitul 0 de la al doilea număr).
- **Efectul de Cascadă (Ripple Carry):** Spre deosebire de un *Half-Adder*, fiecare *Full Adder*

din schemă are o a treia intrare dedicată bitului de transport generat de coloana anterioară (C_{in}). Transportul „se propagă” ca un val sau o undă de la dreapta la stânga (de la cel mai puțin semnificativ bit LSB, până la cel mai semnificativ bit MSB).

- **Depășirea (Overflow):** Dacă adunarea de pe ultima poziție (Bit 3) generează la rândul ei un bit de transport (C_{out}), acesta devine transportul final. În circuitele de calcul, acesta indică o **depășire de ecran (Overflow)**, semnalizând că rezultatul matematic este prea mare pentru a fi reprezentat pe doar 4 biți.

💡 Analogia QualiAdept: „Linia de Asamblare a Găleților”

💡 Sumatorul în cascadă (Ripple Carry Adder):

Imaginează-ți o linie de 8 muncitori (Sumatoarele). Fiecare are sarcina să pună fix 1 litru de apă în sticla din fața lui.

Dar, uneori, din calcule, îi rezultă 2 litri. Pentru că sticla lui are doar 1 litru (Suma 0), el toarnă litrul în plus într-o cană (Carry-ul) și i-o dă fizic muncitorului din stânga lui. Muncitorul din stânga trebuie să aștepte cana vecinului înainte să poată termina propriul său calcul și să dea o cană mai departe.

🔧 Ochiul de Tester: Întârzierea Propagării (Propagation Delay)

🔧 Ca tester de performanță hardware, știi că electricitatea este rapidă, dar nu instantanee. Trecerea curentului prin tranzistorii porților logice durează câteva nanosecunde.

- **Bottleneck-ul Fizic:** Într-un Sumator în cascadă, ultimul bit (bitul 32, să spunem) nu își poate calcula rezultatul final până când „Carry-ul” nu a călătorit prin toate cele 31 de

sumatoare anterioare. Aceasta se numește **Întârzierea Propagării (Propagation Delay)**.

- **De ce e relevant?** Dacă forțezi procesorul (prin Overclocking) să ceară rezultatul înainte ca semnalul să aibă timp fizic să ajungă de la primul la ultimul bit, vei obține un calcul matematic complet greșit. Sistemul se va prăbuși. Acesta este motivul principal pentru care procesoarele au limite maxime de frecvență (GHz). Nu poți grăbi fizica!

Exercițiu de reflexie

Uită-te la a doua diagramă (Sumatorul în cascadă). Ce se întâmplă la ultimul sumator (Bitul 4)? Dacă și acesta generează un *Carry Out* (are 1 în minte), dar nu mai există niciun alt sumator în stânga lui care să îl preia, unde se duce acea informație?

Cum se numește această eroare în lumea IT, atunci când rezultatul unui calcul matematic depășește memoria fizică alocată și este „aruncat” peste margine? (Atenție, am discutat despre ea și în Sesiunea 1, la capitolul Registre!)

Capitolul 6: Algebra Booleană - Matematica din spatele deciziilor

digitale

🧠 Avem tranzistori, avem porți logice, știm să adunăm biți. Dar un procesor modern precum un Intel Core i9 are miliarde de porți logice interconectate. Cum reușesc inginerii să le proiecteze fără să se piardă în acest labirint? Răspunsul este o matematică specială, inventată în anii 1850 (mult înainte de apariția computerelor) de matematicianul englez George Boole: Algebra Booleană.

1. Ce este Algebra Booleană?

Dacă în algebra normală (cea pe care o învățăm la școală) lucrăm cu numere de la $-\infty$ la $+\infty$ și cu operații precum adunarea ($+$) și înmulțirea ($\times$), în algebra booleană lucrăm exclusiv cu două valori: **Adevărat (1)** și **Fals (0)**.

Această algebră folosește porțile logice pe care le cunoaștem, dar le transformă în ecuații matematice:

- **Operația AND (ȘI):** Este scrisă ca o înmulțire: $A \cdot B$ (sau pur și simplu AB).
- **Operația OR (SAU):** Este scrisă ca o adunare: $A + B$.
- **Operația NOT (NEGARE):** Este scrisă cu o bară deasupra: $\overline{A}$ (sau cu un apostrof: A').

Deci, dacă un inginer vrea să spună „Alarma sună (ieșirea) DACĂ senzorul de fum (A) este activ SAU dacă termostatul (B) este activ ȘI butonul de panica (C) este apăsat”, el va scrie simplu ecuația: $Iesire = A + (B \cdot C)$.

2. Optimizarea și Simplificarea - Cum facem procesoare mai ieftine

Scopul principal al algebrei booleene nu este doar de a scrie codul pe hârtie, ci de a-l **simplifica**.

Fiecare poartă logică fizică costă bani, ocupă spațiu prețios pe pastila de siliciu și produce

căldură. Dacă putem folosi o ecuație matematică pentru a reduce numărul de porți fără a schimba rezultatul logic, am creat un procesor mai mic, mai rece și mai rapid!

Legi booleene simple de reducere:

- $A \cdot 0 = 0$ (Ceva AND Fals este mereu Fals - putem șterge poarta).
- $A + 1 = 1$ (Ceva OR Adevărat este mereu Adevărat).
- $A + A = A$ (Dacă avem două semnale identice pe un OR, putem păstra doar unul).

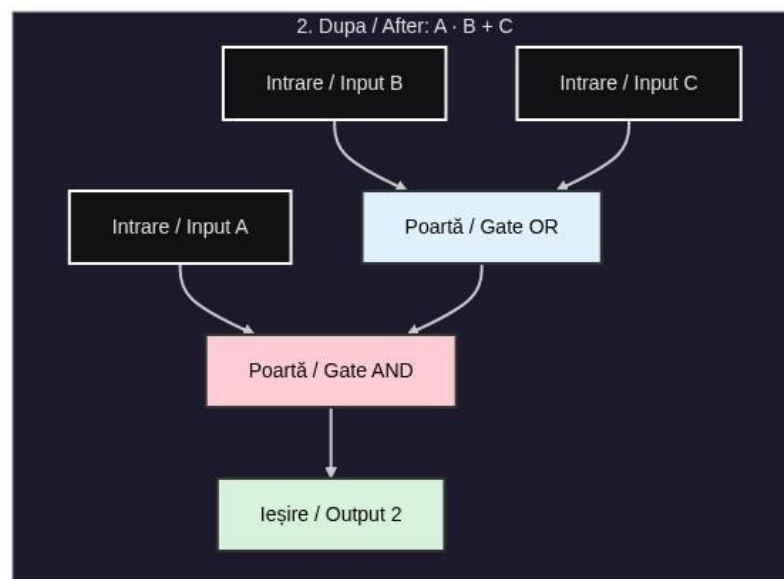
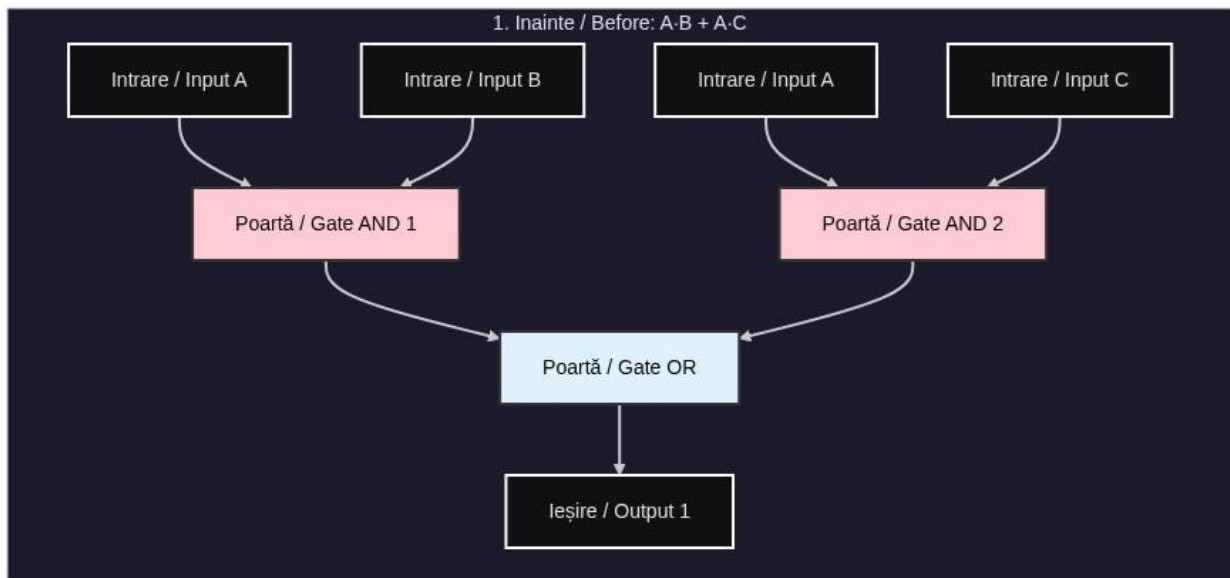


Fig. 12 - Această diagramă ilustrează puterea **Algebrei Booleane** în ingineria hardware, demonstrând cum o funcție logică poate fi simplificată matematic pentru a reduce costurile de producție și consumul de energie, fără a modifica rezultatul final:

- **Circuitul Complex (Stânga):** Implementează direct funcția unificată: $(A \cdot B) + (A \cdot C)$. Acest design necesită **două porți AND** pentru a calcula intersecțiile și **o poartă OR** pentru a le aduna, însumând un total de 3 porți fizice.
- **Procesul de Simplificare:** Folosind legea distributivității (factorizarea matematică), putem extrage factorul comun A . Ecuația devine astfel: $A \cdot (B + C)$.
- **Circuitul Simplificat (Dreapta):** Datorită noii formule, ordinea operațiilor se schimbă. Mai întâi adunăm B și C printr-o **singură poartă OR**, iar rezultatul este înmulțit cu A printr-o **singură poartă AND**.
- **Beneficiul:** Am eliminat complet o poartă AND din hardware. Ambele circuite sunt perfect echivalente din punct de vedere logic (produc exact aceeași tabelă de adevăr), însă cel din dreapta este mai rapid, mai ieftin și consumă mai puțin curent.

3. Legile lui De Morgan - Magia Inversării

Augustus De Morgan a extins munca lui Boole cu două reguli care par pură magie: ele demonstrează că o poartă AND se poate transforma într-o poartă OR (și viceversa) doar folosind porți NOT!

- **Legea 1:** $\overline{A \cdot B} = \overline{A} + \overline{B}$ (Negația lui A ȘI B este egală cu negația lui A SAU negația lui B).
- **Legea 2:** $\overline{A + B} = \overline{A} \cdot \overline{B}$ (Negația lui A SAU B este egală cu negația lui A ȘI negația lui B).

Aceasta este baza logică pentru care am învățat în Capitolul 4 că o poartă **NAND** este o "poartă universală". Inginerii folosesc Legile lui De Morgan pentru a înlocui circuite scumpe și variate cu o simplă mare de porți NAND ieftine!

Analogia QualiAdept: „Regulamentul Clubului”

💡 Circuitul Complex:

La ușa unui club exclusivist, managerul îi dă paznicului următoarele instrucțiuni:

"Lași clientul să intre dacă (Are Cravată ȘI Are Sacou) SAU dacă (Are Cravată ȘI Are Pantofi de piele)." ($Iesire = (Cravata \cdot Sacou) + (Cravata \cdot Pantofi)$).

Paznicul trebuie să verifice cravata de două ori pentru scenariile diferite. E obositor (sunt prea multe porți logice).

💡 Circuitul Simplificat (Algebra Booleană):

Paznicul folosește factorizarea booleană și își dă seama că poate scurta regula fără să o încalce:

"Clientul trebuie neapărat să aibă Cravată ȘI pe lângă asta, (Are Sacou SAU Are Pantofi de piele)." ($Iesire = Cravata \cdot (Sacou + Pantofi)$).

Acum regula este mult mai clară, se aplică mai rapid, iar rezultatul este absolut identic!

🕵️ Ochiul de Tester: Optimizarea Condițiilor de Cod

🕵️ Un QA Engineer care cunoaște algebră booleană are o „superputere” atunci când se uită pe codul sursă scris de programatori (White-box testing).

- **Condiții Redundante (If Statements):** Programatorii scriu adesea instrucțiuni if complicate, de genul:
if ((is_admin == true) OR (is_admin == true AND is_editor == true)) { acces_permis }
- **Rolul Testerului:** Folosind legea de absorbție din algebra booleană ($A + (A \cdot B) = A$), testerul va ști imediat că a doua parte a propoziției este inutilă.

Dacă utilizatorul este deja admin, nu mai contează dacă este și editor. Oricum intră.

- **Reducerea Testelor:** În loc să scrie în Jira 4 scenarii de testare pentru a acoperi toate combinațiile din acel if kilometric, testerul va sugera programatorului să curețe codul: `if (is_admin == true)`. Dintr-o singură privire, am optimizat codul și am salvat ore întregi de testare automată (Condition Coverage)!



Exercițiu de reflexie

Folosind logica umană, încearcă să simplifici următoarea regulă de business dintr-un magazin online:

"Oferim un voucher de reducere DACĂ (Clientul este Nou ȘI Coșul > 100 RON) SAU DACĂ (Clientul este Nou ȘI Coșul <= 100 RON)."

Dacă transformăm asta în Algebră Booleană, ce poartă logică este de fapt complet inutilă în această propoziție? Cu ce regulă foarte simplă ar trebui să o înlocuiască inginerul de software?

Capitolul Bonus: Când matematica binară distruge lumea - Cele mai faimoase bug-uri din istorie 🐞

🕒 Până acum, am privit matematica binară și registrele procesorului ca pe niște concepte abstracte, sigure, închise în laborator.

Dar ce se întâmplă atunci când programatorii uită de limitele fizice ale acestor cutiute digitale? Răspunsul: dezastre financiare, rachete explodate și platforme globale blocate. Să folosim cunoștințele noastre proaspete despre biți, bytes și limite de memorie pentru a diseca 3 dintre cele mai faimoase bug-uri din istoria IT-ului.

1. Explozia de 500 de milioane de dolari (Racheta Ariane 5 - 1996)

La data de 4 iunie 1996, Agenția Spațială Europeană a lansat racheta Ariane 5, un proiect la care se lucrase 10 ani. La doar 37 de secunde de la decolare, racheta a deviat brusc de la traiectorie și s-a autodistrus în aer, vaporizând sateliți în valoare de 500 de milioane de dolari.

- **Care a fost cauza?** Un clasic *Integer Overflow* (Depășire de registru).
- **Explicația tehnică:** Racheta măsura viteza orizontală folosind un număr foarte precis, cu zecimale, pe **64 de biți**. La un moment dat, software-ul a încercat să ia această valoare masivă și să o înghesuie într-un registru mult mai mic, de doar **16 biți** (care poate stoca maxim valoarea 32.767).

- Când racheta a accelerat și viteza ei a depășit valoarea de 32.767, registrul de 16 biți s-a umplut, a dat pe dinafară și a resetat valoarea la un număr complet eronat (negativ). Computerul de bord a crezut că racheta merge cu spatele și a virat brusc motoarele la maximum pentru a „corecta” traiectoria, rupând racheta în bucăți.

2. Sfârșitul universului Pac-Man (Nivelul 256 - 1980)

În jocul arcade original Pac-Man, scopul era să mănânci puncte și să avansezi din nivel în nivel la nesfârșit. Jucătorii profesioniști din anii '80 au descoperit însă ceva bizar: dacă reușeai performanța supraumană de a ajunge la nivelul 256, jumătate din ecran se umplea instantaneu cu caractere aleatorii și gunoaie grafice, făcând jocul imposibil de continuat (cunoscut ca "Kill Screen").

- **Care a fost cauza?** Limita absolută a unui singur Byte (8 biți).
- **Explicația tehnică:** Programatorii, pentru a economisi memoria extrem de scumpă din acea vreme, au stocat contorul nivelului curent într-un singur **Byte**. După cum am învățat în Capitolul 1, un Byte (8 becuțe) poate reține valori de la 0 la 255.
- Când jucătorul termina nivelul 255 (în binar: 11111111) și aduna 1, se producea un overflow. Contorul devenea 00000000 (Nivelul 0). Problema era că algoritmul care desena fructele pe ecran încerca să deseneze „Nivelul 0 minus 1” fructe. A intrat într-o buclă infinită, aruncând în memoria video 256 de caractere aleatorii care au distrus complet ecranul.

3. Cum a stricat Gangnam Style YouTube-ul (2014)

În 2014, hit-ul muzical „Gangnam Style” al artistului sud-coreean PSY a devenit un fenomen global atât de masiv, încât a stricat efectiv contorul de vizualizări al platformei YouTube. Timp de câteva ore, vizualizările videoclipului afișau numere negative.

- **Care a fost cauza?** Limita unui număr întreg pe 32 de biți cu semn (*32-bit signed integer*).
- **Explicația tehnică:** Google a programat contorul de vizualizări folosind un registru de memorie de 32 de biți. Deoarece acest registru folosea primul bit pentru a nota dacă numărul e pozitiv sau negativ (deci îi mai rămâneau 31 de biți pentru valoare), limita maximă absolută pe care o putea reține era **2.147.483.647**.
- Nimeni de la Google nu și-a imaginat că un singur clip va depăși vreodată 2 miliarde de vizualizări. Când Gangnam Style a atins vizualizarea cu numărul 2.147.483.648, bitul de

semn a fost „răsturnat” pe 1 (Negativ), iar contorul a căzut brusc la **-2.147.483.648**. Google a fost forțat să modifice arhitectura codului pe 64 de biți (care are o limită de 9 quintilioane de vizualizări).

💡 Analogia QualiAdept: „Odometrul mașinii”

💡 Toate aceste bug-uri funcționează exact ca odometrul mecanic al unei mașini vechi.

Imaginează-ți că bordul mașinii tale are doar 6 roțițe (poate arăta maxim 999.999 km). Când mașina atinge 1 milion de kilometri, nu apare roțița cu numărul 7. În schimb, toate roțițele se dau peste cap și revin la 000.000.

Dacă tu cumperi acea mașină a doua zi, crezi că este nou-nouță (0 km), când de fapt motorul este complet uzat (1.000.000 km). Calculatorul este la fel de „naiv” ca tine: dacă biții dau pe dinafară, el pur și simplu citește ce a mai rămas din ei.

🕵️ Ochiul de Tester: BVA (Boundary Value Analysis)

🕵️ Ce au în comun aceste trei povești? Ele ne învață esența tehnicii de testare **Boundary Value Analysis (BVA - Analiza Valorilor de Frontieră)**.

- **Mentalitatea de Dezvoltator vs. Tester:** Un programator va scrie codul și va testa o tranzacție cu 10 lei. Totul funcționează perfect. Un tester QA bun știe să privească *sub* cod, direct la memoria alocată.
- **Misiunea QA:** Când un tester vede un câmp, el nu testează doar limita de business (ex: „Nu poți cumpăra mai mult de 10 produse”). El testează **limitele hardware/binare**. Un tester excelent la YouTube în 2010 ar fi creat un script pentru a crește artificial vizualizările unui clip de test la 2.147.483.647 și încă o vizualizare peste, anticipând prăbușirea sistemului înainte ca PSY să o provoace în lumea reală!

🔧 Exercițiu de reflexie: Următorul Sfârșit al Lumii

Digitale

Industria IT se pregătește chiar acum pentru un dezastru masiv care va avea loc pe data de **19 Ianuarie 2038, ora 03:14:07 UTC**.

Acesta este cunoscut sub numele de **Y2K38 (Problema Anului 2038)**.

- *Fă puțin research rapid: Ce contor intern, ascuns adânc în aproape toate sistemele de operare (Unix/Linux), măsoară timpul în secunde începând cu 1 ianuarie 1970? Ce fel de registru binar (câți biți) folosește acesta și de ce se vor bloca brusc milioane de servere în anul 2038 dacă nu sunt actualizate?*